# CSCI2100E Tutorial 3

## Review Your Memory

Hoover H.F. Yin

Department of Information Engineering
The Chinese University of Hong Kong

February 11, 2019

*Poke!*

# Contact


- YIN Ho Fai, Hoover
- Office: Institute of Network Coding (INC), 7/F, SHB
- Email: yhf015 at ie dot cuhk dot edu dot hk

# Outline

1. Memory
2. Pointer
3. Array
4. Function

# Computer Memory

In the view of hardware:

- Basic unit: bit
- Built by memory cells – each store 1 bit

Usually we need multiple bits to represent something...

So in the view of software:

- Basic unit: byte
- Which byte are we reading/writing? Assign each byte an ID!

# Memory Address

Every byte in the memory is assigned a number, which we called memory address

- Variables are stored in memory
- Which memory addresses a variable occupied?

```
int a = 10;
printf("%p\n", &a);
```

- %p stands for pointer
  - Simple speaking, we are printing the input in a specific format
- The operator & is used to get the address of a variable

# Memory Address

The output may change every time you run it...

- It is some kind of security measure...
  - It is called address space layout randomization, which makes it harder to inject malicious code to perform a buffer overflow attack

Here is an example output

**Output**

```
0x7fff0911996c
```

# Memory Address

**Output**

```
0x7fff0911996c
```

Well, how to read it?

- `0x` means it is a hexadecimal number
- The number `7fff0911996c` is the starting address of the variable

In most modern computers, an `int` occupies 4 bytes

- So the `int` variable `a` in the code occupies the address from `7fff0911996c` to `7fff0911996f`

# scanf

Now can you tell the differences between

- `scanf("%d", &a);`
- `scanf("%d", a);`

Suppose a is an `int`, it is non-sense to tell `scanf` the current value of a

- `scanf` is going to overwrite the variable...
- You should tell `scanf` where the does variable locate!
- In this case, we get the address of a by `&a`

If a contains a memory address, the second case means

- `scanf` will write the answer in the variable located at a
- Recall how you `scanf` using %s, you don't need & for an array
  - The name of an array refer to the address of its first element
  - We will discuss more in the section about array

# scanf

Here is an example:

```
int a;
int *b = &a;
scanf("%d", &a);
scanf("%d", b);
```

If you can't read the second line, please first review the lecture notes (or the next section in this tutorial)

In fact, it's one of the ways to 'return' multiple values

- A function can only return one value
- If we give the addresses of variables, the function can write the output directly

# scanf

- If we pass the reference of a variable, it is called pass by reference
  - In C, an address can be treated as a reference, but strictly speaking it is passing a reference by value
- If we pass the value of a variable, it is called pass by value

Usually a function allows the caller not to receive the 'return' value

- Usually passing a `NULL` means that the caller doesn't want to receive the output
- You need to implement it by yourself
  - Don't dereference a `NULL`
  - A minor note: `free()` a null pointer performs nothing by the C specification, i.e., no errors!

Another way to return multiple values?

- Return a `struct`, which can contain multiple members

# Process Memory

Process vs Program

- A program is the executable you compiled from your code
- After you run your program, it is a process
- You can have multiple processes of the same program

Each process has the following memory layout in most modern computers:

| |
|---|
| Stack |
| Heap |
| Data Segment + BSS |
| Text Segment |

# Text Segment

Text segment contains:

- Your code (the machine code), which means functions have addresses
  - We can use function pointers to point a function (will be discuss in the section about function)
  - In fact we can have an address for each instruction (machine code)
  - Your CPU uses a register named program counter to point to the currently running instruction
- Constants (not local constant variables), and string literals
  - Local variables are stored in the stack of the process memory layout
  - A string literal is a string constant, which is hardcoded in your program

# Text Segment

We can print the addresses to verify the layout:

```
const int a = 0;
int main(void)
{
    char *str = "Hello";
    const char str2[] = "Hello";
    printf("%p\n", main); // the address of main()
    printf("%p\n", str); // string literal
    printf("%p\n", &str); // local variable
    printf("%p\n", str2); // const local variable
    printf("%p\n", &a); // constant
    return 0;
}
```

# Text Segment

**Output**

```
0x400530
0x400654
0x7fffc8023e28
0x7fffc8023e20
0x400650
```

- Note that the addresses in the text segment are much smaller than the local variables (which are in the stack)
- The addresses in the stack change every time if the address space layout randomization is on (it's on by default unless you are using a very old OS)
- But note that the addresses in text segment are not randomized

# Text Segment

## Advanced Notes in Computer Science

Did you realize that the stack memory address 0x7fffc8023e20 is nearly 128TB of memory? (1TB = 1024GB)

- In fact, the addresses viewed by a process are not the real physical memory address
- The hardware memory management unit (MMU) performs the mapping of these virtual memory addresses into physical memory addresses
- The translation lookaside buffer (TLB) helps the MMU to speed up the mapping by caching
- We map the memory page by page using page tables, where the size of a page is 4KB in Intel x86 architecture

# Data Segment & BSS

Data segment:

- Contains initialized global and static variables
- The required space already allocated in your executable (as we need to store the initialized value)

BSS:

- BSS stands for Block Started by Symbol
- Contains uninitialized global and static variables
- They are only symbols. The space will be allocated after the process starts

Static variable is the same as global variable!

- Only the compiler cares the difference

# Data Segment & BSS

The following code shows that global and static variables are put very close, i.e., in the same segment

```
int global_data = 1;
int global_bss;
int main(void)
{
    static int static_data = 1;
    static int static_bss;
    printf("%p, %p\n", &global_data, &static_data);
    printf("%p, %p\n", &global_bss, &static_bss);
    return 0;
}
```

# Data Segment & BSS

**Output**

```
0x601034, 0x601038
0x601044, 0x601040
```

See? Initialized global and static variables are put together (data segment), while uninitialized global and static variables are put together (BSS) in a higher address.

How about the file size?

```
int arr[1000000] = {1};
int main(void)
{
    return 0;
}
```

```
int arr[1000000];
int main(void)
{
    return 0;
}
```

# Data Segment & BSS

The executable on the left has size 3.9MB, while the one on the right has size 8.3KB

- The initialized global and static variables in the data segment occupy space in the executable
  - The values are stored in the executable
  - It can be a special case if we initialized the values by 0
    - To optimize the performance, the copy-on-write (COW) scheme may be used if the OS supports
    - A read-only space filled with 0 is pointed
    - The process allocates the space only when you write something into it
    - Try to see the time difference between `calloc()` a huge space, and `malloc()` then `memset()` the space to 0
- The uninitialized global and static variables in the BSS only allocate space after the process starts
  - They are symbols only

# Heap & Stack

Heap:

- A space for dynamic memory allocation, e.g., `malloc()`
  - The size of the heap can be adjusted (`malloc()` will do this for you)
  - Advanced notes: `malloc()` can be implemented to map memory addresses out of the heap region, e.g., before the text segment and after the stack

Stack:

- Grow from higher address to lower address
- Stores all local variables, including local constant variables
- Stores the return value, return address, parameters passed to the function, etc.
- The maximum size for the stack is fixed before the program starts (by the OS)

# Heap & Stack

Now, do you know why you can't create a huge array as a local variable?

- The space for the stack is not enough $\Longrightarrow$ Stack overflow

Why does a deep recurrsion fail?

- Every function call will push some data (return address, parameters, etc.) into the stack
- If the recurrsion is too deep, the space for the stack is not enough $\Longrightarrow$ Stack overflow

This is also a reason why passing/returning an array into a function is just passing/returning the address of the array by design

- The memory in the very old days was very very small
- If you really want to pass/return an array by value, you can wrap it by a `struct`
- But note that copying the values in the array takes time

# Constant?

Local variables are in the stack, so there should be no read-only protection...

```
const int a = 10;
int *b = (int*) &a;
*b = 20;
printf("%d\n", a);
```

We can't change a directly as it's a constant (the compiler stopped it), but can we change it through b? Yes!

- A local constant variable is not constant at all...
- A global constant variable is constant (depends on what OS you are using)

# What is pointer?

Pointer is...

- A variable which store a memory address
- A memory address is just a number

The above 2 points are VERY IMPORTANT!!!

Memory address is just a number...

- There is no concept of 'data type' for an address
  - You can freely do type casting between pointer types
- If I dereference it... '0010101011010110'... How to read it?
  - This is why we have different pointer types

# What is pointer?

The differences between different pointer types are

- The data type if you dereference it
- The pointer arithmetic
  - Addition and subtraction of pointers

+1 to a pointer means add the size of the target type to the address

- +1 to `int*` means +4 to the address (suppose `sizeof(int)` is 4)
- +1 to `char*` means +1 to the address (`sizeof(char)` is 1)

Subtraction is similar, by changing + to −

Oh wait, how about `void*`?

# void*

The property of `void*`

- You can't dereference it!
- In terms of C standard, no pointer arithmetic on `void*` is allowed
  - Meanwhile, most (not all) compilers allow it
  - If it's allowed, +1 to `void*` means +1 to the address
- It acts as an universal pointer type
  - It is a pointer but we don't know what type it is pointing
  - Or... we just don't care...
- Remember, you can freely type cast between pointer types
  - Cast it to another type before you dereference it

# void*

## Historical Notes

There was no `void*` in the very old day. People used `char*` for those type-agnostic pointers. After the introduction of `void*`, there was no pointer arithmetic allowed on it. Most compilers extended and allowed the arithmetic. Basically it's just like a `char*` after extension. It can also prevent you from dereference it wrongly in some complicated statements. You can disable the extension from GCC by adding `-pedantic` as an argument.

# Pointer Arithmetic

Suppose we allow arithmetic on `void*`

```
int a;
int *foo = &a;
char *bar = &a; //may have warnings
char *bar2 = (char*)&a;
void *haha = &a;
printf("Address of a = %p\n", &a);
printf("foo + 1 = %p\n", foo+1);
printf("bar2 + 1 = %p\n", bar2+1);
printf("haha + 1 = %p\n", haha+1);
```

# Pointer Arithmetic

It is okay to assign an `int*` to `char*`

```
char *bar = &a; //may have warnings
```

The compiler warns you because it doesn't sure if you really meant to do it.
To tell the compiler, do a type cast

```
char *bar2 = (char*)&a;
```

# Pointer Arithmetic

Here is an example output. The address may change in different runs.

**Output**

```
Address of a = 0x7fffec2b2324
foo + 1 = 0x7fffec2b2328
bar2 + 1 = 0x7fffec2b2325
haha + 1 = 0x7fffec2b2325
```

As mentioned before

- `foo` is an `int*` so the address +4
- For `char*` and `void*` (if it's allowed), the address +1

# Pointer in C

The star can put left or right

```
int *a;
int* b;
int* c, d; //d is not a pointer
```

The data type of `a`, `b`, `c` is `int*`

- The `*` followed by the variable name indicates it's a pointer
- `d` is not a pointer because `int(* c), d;`

`int *a` means:

- `int *a` : a is a pointer
- `int *a` : which points to an `int`

# Point a Pointer

Can we point to a pointer? Pointer of pointer

```
int a;
int *b = &a;
int **c = &b; //NOT &&a
int ***d = &c;
```

How to understand `int **c`?

- You should read it as `int *(*c)`
  - `int **c` : c is a pointer
  - `int **c` : which points to an `int*`

# Point a Pointer

How about `int ***d`?

- `int ***d` : d is a pointer
- `int ***d` : which points to a pointer
- `int ***d` : which points to an `int*`

Remember don't type `&&a`

- `&a` means the address of the variable `a`, which is a number
- You want to get the address of a number? It's non-sense

Usually `**` is enough in most situations.

# Constant Pointer

```
const int a;
const int *b = &a;
const int * const c = &a;
const int ** const d = &b;
const int * const * const e = &c;
```

The first two are discussed before. How to understand the others?

- `const int * const c` : c is a constant pointer, i.e. c is a constant variable
- `const int * const c` : which points to a `const int`

# Constant Pointer

`const int ** const d`?

- `const int ** const d` : d is a constant pointer
- `const int ** const d` : which points to a `const int*`

`const int * const * const e`?

- `const int * const * const e` : e is a constant pointer
- `const int * const * const e` : which points to a constant pointer
- `const int * const * const e` : which points to a `const int`

If it is possible, don't write this kind of code...

# Pointer to struct

```
struct foo {
    int *x, y;
};
```

Suppose we declare

- `struct foo a;`

Then a occupied a memory of size `sizeof(struct foo)`

- All the members are inside that allocated memory
- `a.x` means the member `x` inside `a`

# Pointer to struct

Suppose we declare

- `struct foo a;`
- `struct foo *b = &a;`

How to dereference `a.x`?

- `*a.x`

How to get `a.y` if we only have `b`?

- `(*b).y`
- `b->y`

Note that the priority of `.` is higher than `*` and `&`
An easy rule to remember

- if the `struct` is a pointer, use `->`
- otherwise, use `.`

# Offset in `struct`

What is the address difference between `&a.y` and `&a`?

- The address difference is called the offset of `y` in `struct foo`
- The offset is a constant, no matter which `struct foo` instance you're measuring
- The offset is decided by the compiler to optimize the performance...
  - Word alignment is a speed up, i.e. hold some unused bytes (paddings) between members
  - In most modern computers, the offset (except `char` and `unsigned char`) is a multiple of 4 bytes

# Offset in struct

How to get the offset?

- `#include <stddef.h>`
  - Stands for "Standard Definitions"
- Use `offsetof`

For example, how to print the offset of `y` in `struct foo`?

```
printf("%lu\n", offsetof(struct foo, y));
```

Can we disable the paddings?

```
#pragma pack(push,1)
struct bar {
    char a; int b;
};
#pragma pack(pop)
```

# union vs struct

What are the differences between

```
union foo {
    char c[4];
    float f;
};
```

```
struct bar {
    char c[4];
    float f;
};
```

- The members in `union` share the same memory space
  - i.e. Same memory space with different names
- Same syntax: use `.` and `->` but replace `struct` by `union`

```
union foo haha;
```

Then, `&haha.c` equals `&haha.f`

# * and &

What are the differences between

- `*&a`
- `&*a`

The order of the operators

- `*&a` first gets the address of `a`, then dereferences it
  - So it is the equivalent to `a`, as long as `a` is a variable
- `&*a` dereferences (treat the value of `a` as an address), then gets the address of it
  - If `a` is not a pointer, you can't do `*a` (compilation error)
  - Well... you can type cast an `int` into a pointer...

The compiler will cancel out the operations if they are valid

# * and &

*a only means the variable located at the address a, but not taking the value out!

```
unsigned long a = 0;
unsigned long b = (unsigned long)&*(unsigned long*)a;
```

It works?

- `(unsigned long)&*(unsigned long*)a` : treat the value of a (i.e. 0) as an address
- `(unsigned long)&*(unsigned long*)a` : get the variable located at address 0
- `(unsigned long)&*(unsigned long*)a` : get the address of the aforementioned variable, i.e. the address is 0
- `(unsigned long)&*(unsigned long*)a` : treat the value 0 as an `unsigned long`

# * and &

Can I get the value from the address 0?

- The address 0 is not owned by you
- Runtime error if you take the value in address 0 out, like

```
//runtime error!
int c = *(int*)0UL; //UL means unsigned long
```

- In fact the definition of `NULL` is `(void*)0`. Verify it by

```
printf("%p\n", NULL);
```

`NULL` is `#define` in many header files like `stdio.h` and `stdlib.h`

# * and &

When will you get the value of a variable? Basically when you really need the value to work on...

- Assign the value to another variable
- Pass by value
- As an array index
- etc...

You may get runtime error if you get the value from a memory location which is not yours

# Change data type

Suppose we have a double

- `double d;`

I want to fill an integer `i` to the first 4 bytes of $d$. Any idea?

- The computer actually doesn't know data types
- The compiler checks you data types
- So fake your compiler :D
- `*(int*)&d = i;`

This skill is useful in network programming

- Still, it is recommended to use `memcpy()` because some architectures (e.g., SPARC) have a very strict word alignment requirement, which may get "bus error" if the address is not aligned (e.g., put an `int` in an address not a multiple of 4)

# What is Array?

In the view of a process...

Array is a continuous memory space!

| | | | | |
|---|---|---|---|---|
| a[0] | a[1] | a[2] | a[3] | a[4] |

# What is Array?

The following code shows the addresses of each element in an array

```
int i;
int a[5];
printf("size of int = %u\n", sizeof(int));
printf("size of a = %u\n", sizeof(a));
for(i = 0; i < 5; i++)
    printf("address of a[%d] = %p\n", i, &a[i]);
```

Now we are going to show that array is a continuous memory space!

# What is Array?

The addresses may change on different runs, but `a` is an `int` array so the differences are always 4, which is the size of `int`

**Output**

```
size of int = 4
size of a = 20
address of a[0] = 0x7fffbf27d6e0
address of a[1] = 0x7fffbf27d6e4
address of a[2] = 0x7fffbf27d6e8
address of a[3] = 0x7fffbf27d6ec
address of a[4] = 0x7fffbf27d6f0
```

Note: `sizeof(a)` gives the size of the array, which is $5 \times$ `sizeof(int)` $= 20$ in the code

# What is 2D-array?

There's no concept of 2D in a series of numbers (memory addresses)... Then what is 2D-array?

- Array of array!

Although we usually draw a table (2D) to express the idea, in fact it's like the following

| a[0] | | a[1] | | a[2] | |
|---|---|---|---|---|---|
| a[0][0] | a[0][1] | a[1][0] | a[1][1] | a[2][0] | a[2][1] |

Because... In the view of a process, array is a continuous memory space!

How about higher dimensional array? Array of array of array of ...

# What is 2D-array?

Let's print out the addresses to verify it!

```
int i, j;
int a[3][2];
printf("size of int = %u\n", sizeof(int));
for(i = 0; i < 3; i++)
    printf("size of a[%d] = %u\n", i, sizeof(a[i]));
printf("size of a = %u\n", sizeof(a));
for(i = 0; i < 3; i++)
    for(j = 0; j < 2; j++)
        printf("address of a[%d][%d] = %p\n", i, j,
                &a[i][j]);
```

# What is 2D-array?

The addresses may change on different runs

**Output**

```
size of int = 4
size of a[0] = 8
size of a[1] = 8
size of a[2] = 8
size of a = 24
address of a[0][0] = 0x7fff5908ae00
address of a[0][1] = 0x7fff5908ae04
address of a[1][0] = 0x7fff5908ae08
address of a[1][1] = 0x7fff5908ae0c
address of a[2][0] = 0x7fff5908ae10
address of a[2][1] = 0x7fff5908ae14
```

# Array Index

When you declare an array...

- You allocate a continuous memory space
- The name of the array refer to the address of its first element

So the two lines in the output of the following program are the same!

```
int a[5];
printf("Value of a      = %p\n", a);
printf("Address of a[0] = %p\n", &a[0]);
```

# Array Index

We can finally explain why the array index start from 0

The following are equivalent:

- `a[i]`
- `*(a+i)`

Your computer can only understand `*(a+i)`

- So there's no out-of-bound check!!

If we don't start the index from 0, then we need to do more additions/subtractions to get the address of the element!

- Every operation takes time!

# Array Index

Addition is commutative... So the following are equivalent:

- `a[i]`
- `*(a+i)`
- `*(i+a)`
- `i[a]`

It means you can write something like

- `2[a]`
- `"Hello World!"[6]`

It's strange, right?

# Array Index

```
int i;
int a[3] = {0,1,2};
for(i = 0; i < 3; i++)
    printf("a[i] = %d\n", a[i]);
for(i = 0; i < 3; i++)
    printf("i[a] = %d\n", i[a]);
```

**Output**

```
a[i] = 0
a[i] = 1
a[i] = 2
```

**Output (cont.)**

```
i[a] = 0
i[a] = 1
i[a] = 2
```

# Negative Index!

If we own the address `a-i`, we can have `a[-i]`!!!

```
int i;
int a[5] = {0,1,2,3,4};
int *b = a+2;
for(i = -2; i <= 2; i++)
    printf("Value of b[%d] = %d\n", i, b[i]);
```

# Negative Index!

**Output**

```
Value of b[-2] = 0
Value of b[-1] = 1
Value of b[0] = 2
Value of b[1] = 3
Value of b[2] = 4
```

Nice!

# Address of an Array

Guess it!

```
int a[4];
printf("%p, %p\n", a, &a);
printf("%p, %p\n", a+1, &a+1);
```

- a is the address of the first element, operations in type `int*`
  - But a itself represent the array, so `sizeof(a)` equals 16
- &a is the address of the array, in type `int(*)[4]`
  - `int(*)[4]` : a pointer
  - `int(*)[4]` : which points to an array with 4 elements
  - `int(*)[4]` : every element in the array is of type `int`

# Address of an Array

The addresses may change on different runs. Note the addresses after addition.

**Output**

```
0x7fffc9d8e5a0, 0x7fffc9d8e5a0
0x7fffc9d8e5a4, 0x7fffc9d8e5b0
```

- `a+1` adds 4 as expected
- The size of the array is 16, so `&a+1` adds 16

# Address of an Array

What are these...?

- `int *x[3];`
- `int (*b)[4] = &a;`
- `int (*c[10])[4]; //c[0] = &a;`
- `int (*(*d)[10])[4] = &c;`

Relax, read it part by part, and don't write them in your code∼

`int *x[3]`

- `int *x[3]` : `x` is an array with 3 elements
- `int *x[3]` : every element is of type `int*`

`int (*b)[4]`

- `int (*b)[4]` : `b` is a pointer
- `int (*b)[4]` : which points to an `int` array with 4 elements

# Address of an Array

`int (*c[10])[4]`

- `int (*c[10])[4]` : c is an array with 10 elements
- `int (*c[10])[4]` : each element is a pointer
- `int (*c[10])[4]` : which points to an array of 4 elements
- `int (*c[10])[4]` :each element is an `int`

`int (*(*d)[10])[4]`

- `int (*(*d)[10])[4]` : d is a pointer
- `int (*(*d)[10])[4]` : points to an array with 10 elements
- `int (*(*d)[10])[4]` : every element is a pointer
- `int (*(*d)[10])[4]` : points to an array of 4 elements
- `int (*(*d)[10])[4]` : every element is an `int`

Golden rule: Read array notation before *

# Array as a Parameter

How to pass a 1D-array?

- In fact you are just passing an address to the function
- Recall that the name of an array refer to the address of its first element

So here are some examples of the function prototype:

- `void func(int arr[5]);`
  - The compiler in fact just skip the size
  - Because we only passed the address, the array size is useless in the operations of `int*`
- `void func(int arr[]);`
- `void func(int *arr);`
  - As we only passed the address of its first element

# Array as a Parameter

How to pass a 2D-array?

```
int main(void)
{
    int i, j;
    int a[3][2];// = {{0,1},{2,3},{4,5}};
    for(i = 0; i < 3; i++)
        for(j = 0; j < 2; j++)
            a[i][j] = i*2+j;
    foo(a);
    return 0;
}
```

I want to pass the 2D array a to the function foo

# Array as a Parameter

```
void *foo(int a[][2])
{
    int i, j;
    for(i = 0; i < 3; i++)
        for(j = 0; j < 2; j++)
            printf("value of a[%d][%d] = %d\n", i, j,
                    a[i][j]);
}
```

The function doesn't know how many items in the first dimension...

# Array as a Parameter

**Output**

```
value of a[0][0] = 0
value of a[0][1] = 1
value of a[1][0] = 2
value of a[1][1] = 3
value of a[2][0] = 4
value of a[2][1] = 5
```

When you pass an array to a function... You don't need to specify the size of the first dimension! Why?

# Array as a Parameter

In fact you can also specify it... But the compiler simply skip it...

Suppose we define

- `int a[3][2];`
- `int *b = (int*) a;`

Then...

- Array is a continuous memory space
- `a[i][j]` means `*(b+i*2+j)` when calculating the address
  - Strictly speaking, it should be `*(*(a+i)+j)` if we know `sizeof(int(*)[2])`
- Computer only understand `*(b+i*2+j)`
- We don't need to know the size of the first dimension
- But we need to know the size of other dimensions!!

# Return a Pointer

Never return the address of a local variable!!!

```
#include <stdio.h>
int *foo(void)
{
    int a = 10;
    return &a;
}
int main(void)
{
    int *a = foo();
    printf("%d\n", *a);
    printf("%d\n", *a);
    return 0;
}
```

# Return a Pointer

The two output lines are different! (The second line may change)

**Output**

```
10
48
```

Why?

- Local variables of a function are stored in the stack
- The memory is deallocated (popped) after the function returned!
- You can point to it, but it can be replaced by other functions
- In fact, the value in `*a` is replaced by `printf`

# Function Pointer

Pointer to a function!! It is possible as the code is in the text segment!

But... What is the data type?

- Replace the function name in the prototype by `(*)`
- For example: `int (*)(int, int)`

How to declare a function pointer?

- The variable name of the pointer is right after the `*`
- For example: `int (*p)(int, int);`

# Function Pointer

Here is an example:

```
int add(int a, int b)
{
    return a+b;
}
int main(void)
{
    int (*p)(int, int) = add;
    printf("%d\n", p(1,2));
    return 0;
}
```

# Function Pointer

Calling `p(1,2)` is equivalent to calling `add(1,2)`

**Output**

3

**Historical Notes**

Do we need `&` and `*` for function pointers? It is too complicated for you to call a function by `(*p)(1,2)` so the C standard allowed you to skip them at the very beginning.

# Function Pointer

Can we cast a function pointer to `void*`?

- Yes, but cast it back before you call it

```
int main(void)
{
    void *p = (void*) add;
    printf("%d\n", ((int (*)(int, int))p)(1,2));
    return 0;
}
```

You will get the same output 3.

# Function Table

How about store the function pointers in an array?

- The array is called a function table
- You can then use a `for`-loop to call them one by one!
- The declaration is similar to a function pointer
- For example: `int (*p[2])(int, int);`
- How to read it?
  - `int (*p[2])(int, int)` : `p` is an array with 2 elements
  - `int (*p[2])(int, int)` : where the array elements are function pointers of type `int (*)(int, int)`

# Function Table

Example:

```
int add(int a, int b)
{
    return a+b;
}
int sub(int a, int b)
{
    return a-b;
}
int main(void)
{
    int (*p[2])(int, int) = {add, sub};
    printf("%d, %d\n", p[0](1,2), p[1](1,2));
    return 0;
}
```

# Return a Function Pointer

How to write the data type of a function pointer as a return data type?

```
int (*func(void))(int, int)
{
    return add;
}
```

How to understand it?

- `int (*func(void))(int, int)` : this is the function signature, which is the name of the function and its parameters
- `int (*func(void))(int, int)` : this is the return data type `int (*)(int, int)`

# Return a Function Pointer

We can store the return value in a function pointer and then call it separately, or

- Call it directly!!

```
printf("%d\n", func()(1,2));
```

Just like the case in function table, after we have the address of the function, we can use the parenthesis to perform a function call.

# Return the Address of a 2D-array

Never return the address of a local variable

- We can return the address of a global 2D-array
- Or the one created by `malloc()`

Still remember what is the data type of the address of a 2D-array?

```
int a[2][3] = {0, 1, 2, 3, 4, 5};
```

- `int(*)[3]`

Then, how to write the function?

```
int (*func(void))[3] { return a; }
```

- `int (*func(void))[3]` : the function signature
- `int (*func(void))[3]` : return the data type `int(*)[3]`

# Return the Address of a 2D-array

How to store the return value?

- Create a variable of type `int (*)[3]`

```
int (*p)[3] = func();
```

With the similar idea, how to use `malloc()` to create a 2D-array?

```
int (*arr)[3] = (int(*)[3]) malloc(2*3*sizeof(int));
```

But...

- We seldom really pass or `malloc()` a 2D-array
- We used to treat the array as a 1D-array and calculate the index by ourselves

# typedef

It is difficult to read the code which returns a function pointer or the address of a 2D-array.

- We can use `typedef` to make the code more readable

```
typedef int (*bar)(int, int);
bar func(void)
{
    return add;
}
```

```
typedef int (*bar)[3];
bar func(void) { return a; }
```

# Function Prototype

Back to the basic. What is the usage of the function prototype? Suppose we have two C files.

```
#include <stdio.h>

//extern float foo(int);

int main(void)
{
    float b = foo(2.71828);
    printf("%f\n", b);
    return 0;
}
```

```
#include <stdio.h>

float foo(int a)
{
    printf("%d\n", a);
    return 3.14159f;
}
```

# Function Prototype

If you still remember...

- We can compile two files independently into object files (preprocessor → compiler → assembler)
- And then, the linker links object files together into an executable

It means... you don't know how to 'use' the function!

- What are the data types of the parameters? Should we do type casting?
- Is there any return value? What is the data type?

Well, let's run it and see the result!

**Output**

```
1
1078530048.000000
```

# Function Prototype

The output is not as expected, right? If we uncomment the line

```
//extern float foo(int);
```

and compile the files, we will get

**Output**

```
2
3.141590
```

With the prototype

- We know how to read the bytes we passed and returned

Without the prototype

- All the values are assumed to be `int`

Note: In fact the keyword `extern` is for human (indicates it is in another file), the compiler will simply skip it.

# Minimal Function Declaration

Does this code work?

```
#include <stdio.h>
foo()
{
    puts("Foo");
}
main()
{
    int a = foo(10);
}
```

- `foo()` seems take no parameter but we passed an `int` to it
- `foo()` returns nothing but we get a return value from it
- We don't have return value in `main()`

# Minimal Function Declaration

The code works!!!

- You don't write it down, the compiler won't check

What is the difference between `void foo(void)` and `void foo()`?

- `void foo(void)` : You can't pass parameters because the compiler will stop you
- `void foo()` : You can pass parameters because the compiler won't check it

Why do we have this feature in C?

- C was built from another language called B, step by step
- This is the way to declare function in B

# return 0

Why return 0?

```
#include <stdio.h>

int main(void)
{
    printf("Hello World!\n");
    return 0;
}
```

- It means `exit successfully` (no error)
- There can be many kinds of error but we should have one kind of success

Oh, our program exited successfully... then what?

# return 0

Where does the return value go?

- Why `main()` is executed? There must be something called our `main()`!
- The return value will finally pass to the process which call our program

To get the return value, different shells have different way... Here's some examples

**bash**

```
echo $?
```

**csh**

```
echo $status
```

**cmd (Windows)**

```
echo %ERRORLEVEL%
```

- `echo` is self-explanatory: repeat what you said
- The shell actually substitute the 'variable' part before calling `echo`

# return 0

It is used when you need to 'chain' programs (like preprocessor, compiler, assembler, linker)

- Why you need to run the next part if it has already failed?

Suppose we have 3 programs:

- `return0` : A program outputs nothing and return 0 in `main()`
- `return1` : A program outputs nothing and return 1 in `main()`
- `hello` : A program prints Hello on screen

**Execution Command**

```
./return0 && ./hello
```

**Output**

```
Hello
```

**Execution Command**

```
./return1 && ./hello
```

**Output (Nothing)**

# `return 0`

The execute commands mean:

- First execute `./return0` (or `./return1`)
- If it returns 0, then run `./hello`
  - So `./hello` is not executed after `./return1`

The && is similar to that in C (logical AND). Beware:

- 0 means false in C
- 0 means true in shells

# return 0

What will we get if we don't have a return value in `main()`?

```
void main(void){ printf("foo\n"); }
```

- You will get the return value 4
- It just the value in the corresponding location of the stack
- In this example, the value 4 is the return value of `printf()`
- In fact the return data type of `main()` in the function prototype is not so important: the shell only gets an `unsigned char` in Mac / Linux while it gets an `int` in Windows
- Still, the return data type affect the way to store the return value. For example, you can't get 3 if you return a `double` 3 because the shell treats it as an integer but in fact it is in the IEEE754 format

# Dynamic Number of Arguments

We can use a dynamic number of arguments in `printf()` and `scanf()`. How to write this kind of variadic functions?

- Use `...` in the function prototype
- `#include <stdarg.h>`
- The only requirement: you need at least one named argument
  - Both `printf()` and `scanf()` have a formatted string as the named argument

Let's have an example. We want to pass an `int` 3, a `double` 3.14, and a string `Hello`.

```
int main(void)
{
    func(0, 3, 3.14, "Hello");
    return 0;
}
```

# Dynamic Number of Arguments

We first have a look on how to write the function body.

```
void func(int n, ...)
{
    va_list vl; // create a variable argument list
    va_start(vl, n); // initialization
    int i = va_arg(vl, int);
    double d = va_arg(vl, double);
    char *s = va_arg(vl, char*);
    va_end(vl); // end using the list
    printf("%d, %f, %s\n", i, d, s);
}
```

We passed the variable name of the last named argument, which is `n` in this example, to the macro function `va_start()`.

# Dynamic Number of Arguments

We don't know how many arguments are passed into the function

- The first argument can be used to provide some hints

To get the arguments...

1. Declare the variable argument list
2. Use `va_start()` to initialize the list (it's a macro function)
   - All the arguments are pushed in the stack, so we can get the remaining arguments after we know where is the last named argument
3. Use `va_arg()` to get the arguments, but we need to know the data type beforehand (again, it is a macro function)
   - `float` in the `...` part are casted to `double` automatically
   - That's why we use `%f` for both `float` and `double` in `printf`
4. When you finish using the list, use `va_end()` to finish the task (macro function again)

# The Arguments of `main()`

Still remember there can be 3 arguments in `main()`? We have the following commonly used function signatures for `main()`

- `main(void)`
- `main(int argc, char **argv)`
- `main(int argc, char **argv, char **envp)`

Note: We can change `char **argv` into `char *argv[]` as they are the same in function arguments (similar for `char **envp`)

The 3 arguments are

- `argc` : argument count, the number of arguments
- `argv` : argument vector, an array of strings storing the arguments
- `envp` : environment pointer, an array of strings storing the environment variables

# The Arguments of `main()`

The environment variables provide information from your environment like your login name, the current path, etc.

- How many items in the array `envp`? We don't know, but we know the last item is `NULL`
- Most but not all compilers support `envp` as an argument in `main()`. The C standard way to get an environment variable is to `#include <stdlib.h>` and use `getenv()`

How about the first 2 arguments?

- They are useful in command-line interface (CLI), where we need to type the name of the executable to run a program.

# The Arguments of `main()`

For example, when we use the GCC compiler to compile the C source file `main.c` and output the executable named `main`, we can use the command:

```
gcc main.c -o main
```

- `gcc` is the program name of the compiler GCC
- `main.c -o main` are the arguments for GCC, which instructs it to do the compilation
- The whole command is tokenized (separated by spaces) and considered as the arguments for `main()`
- So the arguments passed to GCC in the above example are
  1. `gcc`
  2. `main.c`
  3. `-o`
  4. `main`

# The Arguments of main()

Here is an example to print all the arguments:

```
#include <stdio.h>

int main(int argc, char **argv)
{
    int i;
    for(i = 0; i < argc; i++)
        puts(argv[i]);
    return 0;
}
```

# Summary

- Memory: Memory address is a number
- Process Memory: 4 sections: text segment, data segment & BSS, heap and stack
- Text Segment: Contains the machine code and (non-local) constants
- Data Segment: Contains initialized global and static variables
- BSS: Contains uninitialized global and static variables
- Heap: For dynamic memory allocation
- Stack: All local variables, function return values, return addresses, function parameters (arguments) are stored here
- Stack Overflow: Occur when the space for the stack is not enough, e.g., create a huge array as a local variable, or a very deep recurrsion

# Summary

- Pointer: Pointer is a variable which stores a memory address
- Pointer Arithmetic: Add 1 to a pointer will add the size of the target data type to the address
- Pointer Type Casting: You can freely cast a pointer type to another, without changing the content
- Structure: `a->x` is equivalent to `(*a).x`
- Array: In the view of a process, array is a continuous memory space. The variable name of an array refer to the address of its first element
- 2D Array: Array of array, a continuous memory space (in the view of a process)
- Array Index: `a[i]` is equivalent to `*(a+i)`

## Summary

- Local Variables: Local variables deallocate after the function returned. So never return the addresses of local variables
- Function Pointer: A pointer which points to a function
- Function Prototype: To tell the compiler how to use the functions
- Function Declaration: If you don't state the return data type and/or `void` in the function signature, the compiler simply won't check it
- `return 0` in `main()`: To indicate the process ended successfully
- Variadic Functions: Dynamic number of arguments. Use `...` in the function prototype
- Arguments in `main()`: To get the arguments when you start the program